

Оптимизации выполнения: ускорение конвейера

Предсказание, спекуляция, переупорядочивание и
многопоточность

Содержание

- 1. Введение: почему обычный pipeline перестаёт ускоряться
- 2. Предсказание условных переходов
- 3. Спекулятивное выполнение
- 4. Внеочередное выполнение команд
- 5. Многопоточность
- 6. Сравнение подходов
- 7. Заключение: почему следующим узким местом становится память

1. Введение: почему обычный pipeline перестаёт ускоряться

Идея pipeline

Pipeline ускоряет выполнение не одной инструкции, а потока инструкций:

- пока одна инструкция в EX ,
- следующая может быть в ID ,
- ещё одна — в IF .

В идеальном случае после заполнения конвейера получаем:

- примерно 1 завершённую инструкцию за такт.

Но на практике pipeline регулярно простаивает.

1. Введение: почему обычный pipeline перестаёт ускоряться

Что ломает идеальную картину

Три главные причины остановок:

1. Условные переходы

Мы не знаем заранее, какой `PC` будет следующим.

2. Зависимости между инструкциями

Следующая инструкция ждёт результат предыдущей.

3. Долгий доступ к памяти

Даже если ALU быстрая, память может тормозить весь поток.

Итог: чем глубже pipeline, тем заметнее цена каждой ошибки и каждого ожидания.

1. Введение: почему обычный pipeline перестаёт ускоряться

Почему нельзя просто «сделать pipeline глубже»

Более глубокий pipeline часто даёт:

- меньшую работу за стадию;
- более высокую частоту;
- но и **большую цену остановки.**

Если branch разрешается поздно, то при неверном пути приходится очищать больше стадий.

То есть:

глубокий pipeline без дополнительных техник ускорения начинает сам себе мешать.

1. Введение: почему обычный pipeline перестаёт ускоряться

Где заканчивается ISA и начинается микроархитектура

Одна и та же программа RV32I может выполняться на разных ядрах:

- простое in-order ядро;
- pipeline-ядро с branch prediction;
- более сложное out-of-order ядро.

С точки зрения ISA инструкции те же самые.

Меняется не **что** выполняется, а **как именно ядро организует исполнение**.

2. Предсказание условных переходов

Почему branch мешает pipeline

Для обычной инструкции следующий адрес прост:

- $PC_{next} = PC + 4$

Для условного перехода нужно сначала узнать:

- истинно ли условие;
- какой адрес цели.

Пока это неизвестно, стадия **IF** не понимает, откуда брать следующую инструкцию.

Иначе говоря, branch создаёт **control hazard**.

2. Предсказание условных переходов

Самая простая реакция — ждать

Можно ничего не угадывать:

- встретили `branch`;
- остановили `fetch`;
- дождались результата сравнения;
- только потом выбрали следующий `PC` .

Это просто, но плохо для производительности:

- `pipeline` простаивает,
- `IPC` падает,
- особенно в коде с циклами и `if` .

2. Предсказание условных переходов

Идея branch prediction

Вместо ожидания процессор предсказывает:

- будет переход или нет;
- иногда сразу и адрес цели.

Тогда pipeline продолжает fetch так, как будто ответ уже известен.

Если предсказание верное — почти не теряем такты.

Если неверное — придётся очищать неверный путь.

2. Предсказание условных переходов

Статическое и динамическое предсказание

Статическое:

- всегда считать branch not taken;
- или, например, backward taken / forward not taken.

Динамическое:

- смотреть, как этот branch вёл себя раньше;
- хранить историю и использовать её снова.

Главная идея динамики:

поведение переходов в программах часто повторяется.

2. Предсказание условных переходов

1-bit predictor

Для каждого branch можно хранить 1 бит:

- 1 — в прошлый раз переход был taken;
- 0 — в прошлый раз был not taken.

На следующем проходе предсказываем то же самое.

Плюс:

- очень просто.

Минус:

- branch на границе цикла часто даёт две ошибки подряд:
 - один раз на выходе из цикла,

2. Предсказание условных переходов

2-bit saturating counter

Часто используют 2-битное состояние:

- strongly not taken
- weakly not taken
- weakly taken
- strongly taken

Идея:

- одно редкое отклонение не должно сразу полностью переворачивать предсказание.

Это особенно полезно для циклов:

2. Предсказание условных переходов

Что обычно нужно предсказать на практике

Часто мало понять только `taken/not taken`.

Нужно ещё быстро узнать:

- куда переходить, если он `taken`;
- то есть `target address`.

Поэтому в микроархитектуре рядом с `predictor` часто появляются структуры вроде:

- таблицы истории `branch`;
- буфера целей перехода.

Для `pipeline` важно одно:

`fetch` хочет получить правильный следующий `PC` как можно раньше.

3. Спекулятивное выполнение

Предсказание само по себе ещё не ускоряет

Если процессор предсказал branch, но всё равно ничего не делает дальше, то выигрыша почти нет.

Ускорение появляется, когда ядро начинает:

- fetch по предсказанному пути;
- decode следующих инструкций;
- а иногда и выполнять их заранее.

Это и есть **speculative execution**.

3. Спекулятивное выполнение

Что значит «спекулятивно»

Процессор действует так, как будто предсказание уже точно верное.

Но есть важная оговорка:

- результаты пока нельзя считать окончательными,
- потому что branch может оказаться предсказан неверно.

То есть спекулятивные инструкции:

- могут продвинуться далеко по pipeline,
- но ещё не обязаны менять архитектурное состояние навсегда.

3. Спекулятивное выполнение

Если предсказание оказалось неверным

Тогда нужно:

1. отбросить инструкции с неверного пути;
2. восстановить правильный `PC` ;
3. начать fetch заново.

На уровне pipeline это выглядит как:

- **flush** неверных стадий;
- потеря нескольких тактов;
- повторное заполнение конвейера.

Чем глубже pipeline, тем дороже misprediction.

3. Спекулятивное выполнение

Почему важен commit

Нужно различать два состояния:

- инструкция уже вычислена внутри ядра;
- инструкция уже официально изменила архитектурное состояние.

`Commit` — это момент, когда результат считается окончательным:

- можно записать его как «настоящий»;
- после этого откатывать уже нельзя.

Идея commit особенно важна для out-of-order ядер.

4. Внеочередное выполнение команд

Почему in-order execution тормозит

Рассмотрим поток:

```
lw    t0, 0(a0)
add   t1, t0, t2
mul   t3, t4, t5
sub   t6, t7, t8
```

Если `lw` долго ждёт память, то простое in-order ядро может остановить продвижение следующих инструкций, даже если `mul` и `sub` от `lw` не зависят.

Получается простой: независимая работа есть, но ядро её не использует.

4. Внеочередное выполнение команд

Идея out-of-order

Процессор пытается:

- декодировать поток инструкций по порядку,
- но исполнять готовые инструкции не обязательно по порядку.

Если инструкция:

- уже получила операнды,
- и нужный исполнительный блок свободен,

то её можно запускать раньше другой, которая всё ещё ждёт данные.

Так ядро скрывает часть задержек.

4. Внеочередное выполнение команд

Какие зависимости мешают переупорядочиванию

Три классических типа зависимостей:

- **RAW** — Read After Write
Настоящая зависимость по данным.
- **WAR** — Write After Read
Конфликт по порядку использования имени регистра.
- **WAW** — Write After Write
Конфликт по порядку записей в один и тот же регистр.

Для корректности обязательно сохранять смысл программы, даже если внутри ядра исполнение идёт иначе.

4. Внеочередное выполнение команд

Register renaming

Проблема WAR и WAW часто связана не с настоящей зависимостью по данным, а с тем, что разные инструкции используют **одно и то же архитектурное имя регистра**.

Идея renaming:

- архитектурных регистров видно, например, 32;
- внутри ядра может быть больше **физических** регистров;
- новое значение **x5** временно кладётся в новый physical register.

Так убираются ложные конфликты по именам.

4. Внеочередное выполнение команд

Упрощённая схема ОоО-ядра

Обычно логика выглядит так:

1. **Fetch / Decode** — берём инструкции по порядку.
2. **Rename** — выдаём новые физические регистры.
3. **Issue / Schedule** — ждём готовности операндов.
4. **Execute** — запускаем готовые инструкции.
5. **Write result** — результат становится доступен зависимым.
6. **Commit** — архитектурное состояние обновляется в правильном порядке.

Ключевая мысль:

исполнение может быть вне порядка, а commit — обычно по порядку.

4. Внеочередное выполнение команд

Зачем нужен reorder buffer

Если инструкции исполняются не по порядку, нужен механизм, который позволит:

- сохранить результаты временно;
- дождаться, когда старшие инструкции завершатся;
- зафиксировать всё в правильном программном порядке;
- корректно отменить спекулятивную работу при ошибке branch.

Эту роль обычно играет **ROB** — **reorder buffer**.

Он помогает совместить:

- агрессивное исполнение,

4. Внеочередное выполнение команд

Что мы выигрываем от OoO

OoO особенно полезен, когда в потоке есть:

- долгие load;
- инструкции с разной латентностью;
- несколько независимых операций подряд.

Тогда ядро может:

- не ждать самый медленный участок слепо;
- а выполнять всё, что уже готово.

Цена за это — заметно более сложная микроархитектура.

5. Многопоточность

Идея многопоточности в ядре

Если один поток временно не может двигаться дальше, можно дать шанс другому.

Многопоточность помогает скрывать:

- branch stalls;
- memory latency;
- ожидание операндов.

Это уже другой способ загрузить pipeline полезной работой.

5. Многопоточность

Coarse-grained multithreading

Ядро некоторое время работает с одним потоком, а при серьёзной задержке переключается на другой.

Плюсы:

- проще, чем очень агрессивное параллельное исполнение.

Минусы:

- между переключениями поток всё равно может простаивать;
- часть состояния нужно хранить для нескольких потоков.

5. Многопоточность

Fine-grained multithreading

Ядро может чередовать потоки чаще, например почти каждый такт.

Идея:

- если один поток упёрся в зависимость,
- на следующем такте можно выбрать другой.

Такой подход хорошо прячет латентность, но требует аккуратной организации fetch/decode и хранения контекста потоков.

5. Многопоточность

SMT — simultaneous multithreading

Здесь ядро пытается держать в работе инструкции из **нескольких потоков одновременно**.

То есть разные исполнительные блоки в одном и том же такте могут получать инструкции разных потоков.

Идея похожа на OoO:

- не оставлять ресурсы ядра пустыми,
- если есть другая полезная работа.

Но аппаратная сложность растёт ещё сильнее.

5. Многопоточность

Что ближе FPGA-разработчику

Для FPGA и учебных ядер чаще реалистичны:

- простое in-order ядро;
- pipeline с умеренным branch handling;
- иногда coarse/fine-grained multithreading как идея скрытия ожиданий.

Полноценный OoO + SMT — это уже заметно более тяжёлый класс дизайна:

- больше логики;
- больше буферов и таблиц;
- сложнее verification и timing closure.

6. Сравнение подходов

Какой метод что именно лечит

Подход	Что пытается скрыть	Главная идея
Branch prediction	control hazards	заранее угадать следующий PC
Speculative execution	потери после branch	работать вперёд по предсказанному пути
Out-of-order execution	ожидание данных и разную латентность	выполнять готовые инструкции раньше
Multithreading	простои из-за stalls и memory latency	переключиться на другой поток

Важно: эти техники не исключают друг друга, а часто работают вместе

6. Сравнение подходов

Что растёт вместе с производительностью

Вместе с ускорением обычно растут:

- площадь логики;
- энергопотребление;
- число внутренних структур состояния;
- сложность отладки;
- сложность верификации.

Поэтому вопрос обычно звучит не так:

- «какая техника самая умная?»

а так:

6. Сравнение подходов

Интуитивная лестница сложности

Условно можно думать так:

1. Простой pipeline
2. Pipeline + branch prediction
3. Pipeline + speculation
4. OoO execution
5. OoO + multithreading / SMT

Каждая ступень пытается всё лучше использовать доступный parallelism, но требует всё более сложной микроархитектуры.

7. Заключение: почему следующим узким местом становится память

Даже умное ядро не всесильно

Предсказание, спекуляция и ОоО помогают лучше загружать pipeline.

Но есть фундаментальная проблема:

- ALU и pipeline могут работать очень быстро,
- а память часто остаётся сравнительно медленной.

Если данных нет рядом с ядром,
то даже очень умный процессор начинает ждать.

7. Заключение: почему следующим узким местом становится память

Во что упирается ускорение дальше

Дальше на первый план выходят вопросы:

- где лежат данные;
- почему нужен cache;
- как виртуальный адрес превращается в физический;
- как ограничивается доступ к памяти;
- как процессор реагирует на внешние события и прерывания.

То есть следующий слой сложности — уже не только pipeline, а процессор внутри полноценной вычислительной системы.

7. Заключение: почему следующим узким местом становится память

Главное из лекции

- Простой pipeline ускоряет поток инструкций, но быстро упирается в stalls.
- Branch prediction уменьшает потери на условных переходах.
- Speculation позволяет идти вперёд до окончательного ответа.
- OoO использует независимость между инструкциями.
- Multithreading помогает не оставлять pipeline пустым.
- Чем умнее микроархитектура, тем выше её сложность.
- Следующий большой bottleneck — память и system-level механизмы.

Следующая лекция

Память, трансляция адресов, защита и прерывания в RISC-V